

Programming Assignment 1

Union-Find

Track A: Implementation and Empirical Study

Student	Jack McCowan
Student ID	24950750
Course	41052 – Advanced Algorithms
Instructor	Troy Lee
Submitted	30/09/2026
Repository	https://github.com/jackmccowan/assignment1
Video	https://youtu.be/cQZLbVFKMuQ

Contents

1. What I built	2
1.1 The data structure.....	2
1.2 Design decisions.....	2
1.3 Variants and their theoretical bounds	2
1.4 Application: maze generation	3
1.5 Navigating the code	3
1.6 Correctness evidence.....	3
2. Empirical study.....	5
2.1 Research questions	5
2.2 Hypotheses	5
2.3 Experimental setup	6
2.4 Results.....	8
2.5 Analysis	13
2.6 Limitations and further work.....	14
3. What I learned	15
3.1 What surprised me.....	15
3.2 What I initially got wrong.....	15
3.3 What I understand now	15
4. AI use.....	17
4.1 Tools used and how much	17
4.2 What I used them for	17
4.3 Where the AI was wrong, unhelpful, or misleading	17
4.4 What I understood versus what I took on trust.....	18
References	19
Appendix A: Reproducing the results	19
Appendix B: Hypotheses	20

1. What I built

1.1 The data structure

A disjoint-set (union-find) structure maintains a partition of the elements $0 \dots n-1$ into sets and supports `find(x)` (return a representative of x 's set), `unite(a, b)` (merge the two sets) and `connected(a, b)`. It is represented as a forest: each element stores a parent index, and each root is the representative of its tree (Galler & Fischer 1964). The two classical improvements are a linking rule that keeps trees shallow (union by rank or by size) and a path rule that shortens paths during find (path compression or path halving). An early analysis of the combination gave $O(\log^* n)$ per operation (Hopcroft & Ullman 1973). The sharper result is that m operations cost $O(m \alpha(n))$ amortized, where α is the inverse Ackermann function (Tarjan 1975; Tarjan & van Leeuwen 1984). This bound is tight in the cell-probe model (Fredman & Saks 1989), and $\alpha(n) \leq 4$ for any n of practical size (Cormen et al. 2022).

1.2 Design decisions

- **Language and build.** C++17, header-only, built with CMake. Tests run in a Debug build with AddressSanitizer (and UndefinedBehaviorSanitizer on GCC/Clang). Benchmarks run in a Release build (/O2 on MSVC, -O2 -march=native on GCC/Clang).
- **Variants chosen at compile time.** `UnionFind<Link, Path>` takes the linking rule (naive, by rank, by size) and the path rule (none, full compression, halving) as template parameters, giving nine combinations. Each combination is compiled separately, so the timed loop contains no branch on the variant. A separate `QuickFind` class is the baseline.
- **Representation.** A `std::vector<uint32_t>` of parents and, for rank or size linking, a second array for rank or size. Naive linking does not allocate the second array, so it uses half the memory of the other variants. 32-bit indices limit n to $2^{32} - 1$ and halve memory traffic compared with 64-bit indices.
- **Iterative finds.** Every find is a loop. Full compression uses two passes (find the root, then repoint the path) instead of the recursive textbook version (Cormen et al. 2022), because naive linking can build a path of length $n - 1$, which would overflow the call stack.
- **Instrumentation without timing cost.** A third template parameter, `Counting`, adds counters for finds, path length, parent writes and maximum path. Its code is under `if constexpr`, so the timed build contains none of it. Counts are collected in a separate, untimed run.
- **Preconditions.** Indices must be less than n . This is checked by `assert` in Debug builds and not checked in Release builds, which is the usual trade-off for a performance-critical structure.

1.3 Variants and their theoretical bounds

Bounds are for m operations on n elements with $m \geq n$. A worst-case bound on a single operation and an amortized bound per operation are different claims: with path compression, a single find can still be expensive but pays for later ones.

Variant (link / path)	Worst-case single operation	Amortized per operation
Quick-find	find $O(1)$, unite $\Theta(n)$	$\Theta(n)$ per unite
Naive / none	$O(n)$	$O(n)$
Naive / compression	$O(n)$	$O(\log_{1+m/n} n)$ (Tarjan & van Leeuwen 1984)
Naive / halving	$O(n)$	$O(\log_{1+m/n} n)$ (Tarjan & van Leeuwen 1984)

Variant (link / path)	Worst-case single operation	Amortized per operation
Rank or size / none	$O(\log n)$	$O(\log n)$ (Cormen et al. 2022)
Rank or size / compression	$O(\log n)$	$O(\alpha(n))$ (Tarjan 1975; Tarjan & van Leeuwen 1984)
Rank or size / halving	$O(\log n)$	$O(\alpha(n))$ (Tarjan & van Leeuwen 1984)

Table 1: Variants under study.

1.4 Application: maze generation

The application is maze generation by a randomised version of Kruskal’s algorithm (Kruskal 1956). A $w \times h$ grid has one element per cell, and a wall between each pair of adjacent cells. The walls are visited in random order, and a wall is removed whenever unite on its two cells succeeds. The result is a spanning tree of the grid (a “perfect” maze):

- **No cycles:** a wall is removed only if its cells were in different sets, so each removal joins two trees.
- **Connected:** the grid is connected, so while two components remain, some wall between them is still to be examined.

Together these give exactly $w \cdot h - 1$ removed walls. The generator (`include/uf/maze.hpp`) contains no randomness. The caller shuffles the walls, so the benchmark can do so before timing.

1.5 Navigating the code

File	Contents
<code>include/uf/union_find.hpp</code>	<code>UnionFind<Link, Path, Counting></code> : the data structure
<code>include/uf/quick_find.hpp</code>	Quick-find baseline
<code>include/uf/variants.hpp</code>	The single list of variants iterated by tests and benchmarks
<code>include/uf/maze.hpp</code>	<code>grid_walls()</code> and <code>carve<UF>()</code> : maze generation
<code>tests/test_union_find.cpp</code>	Edge cases, randomised oracle, invariants, counters
<code>tests/test_maze.cpp</code>	Spanning-tree checks for every variant
<code>bench/bench_main.cpp</code> , <code>bench/platform.hpp</code>	Benchmark harness; CPU pinning
<code>demo/demo_main.cpp</code>	Prints the parent array step by step, or draws a maze
<code>scripts/plot.py</code>	Reads <code>results/bench*.csv</code> , writes <code>report/figures/</code>

Table 2: Repository layout.

1.6 Correctness evidence

- **Oracle comparison.** For every variant, 300 random unions on 64 elements (5 seeds) are checked after each step against component labels recomputed by breadth-first search, which does not use union-find.
- **Invariants.** After 2,000 random operations on 1,000 elements (5 seeds): every element reaches a root; with rank linking, rank strictly increases from child to parent and never exceeds $\log_2 n$; with rank or size linking, depth never exceeds $\log_2 n$; with size linking, each root’s stored size equals its member count.

- **Counters.** Hand-computed counts on a 5-element chain for each path rule, and a randomised check that each find adds exactly the pre-find depth to the path length and that counting does not change any answer.
- **Maze.** For every variant and several grid shapes (including 0×0 , 1×1 and one-cell-wide corridors), the removed walls must join adjacent cells, number $w \cdot h - 1$, and connect every cell (checked by breadth-first search). All variants must carve the identical maze. As a check on the test itself, removing the `if` around `unite` in `carve` makes it fail for every variant. The failure is caught by the connectivity check, not the wall count.
- **Benchmark cross-check.** Each run folds every answer into an order-sensitive checksum. All variants, and the counting build, must produce the same checksum on the same operations, or the harness stops.
- **Deep paths.** `unite(i, i+1)` on 200,000 elements under naive linking builds a path of length $n - 1$. The iterative finds handle it without stack overflow.

2. Empirical study

2.1 Research questions

- **RQ1.** Once path compression is used, does the choice between rank and size linking matter?
- **RQ2.** Path halving or full compression: halving makes one pass but more writes. Which wins, and does that change with n ?
- **RQ3.** $\alpha(n)$ is effectively constant. Is the measured time per operation constant too, and if not, why?
- **RQ4.** In an application (Kruskal's algorithm), does the union-find variant matter once edges must be sorted?
- **RQ5.** Where do theory and practice agree, i.e. where do the textbook growth rates show up directly?

Patwary et al. (2010) compare a wider range of union-find variants experimentally. This study focuses on explaining the gap between the bounds and measured time, using operation counts and a controlled change of memory layout.

2.2 Hypotheses

The hypotheses were recorded before the full benchmark runs. Each is stated with its reasoning, the result that would refute it, and the experiment that tests it, in the accompanying hypotheses document (Appendix B). Table 3 summarises them.

#	Recorded	Prediction (summary)
H1	2026-09-28	rank_compress and size_compress (and rank_half and size_half) are within 10% of each other at every n .
H2	2026-09-28, revised 2026-09-29	Halving and full compression are within 15% of each other at $n \geq 2^{22}$; any gap shrinks as n grows.
H3a	2026-09-28, revised 2026-09-29	For rank_compress, path length per find stays flat from 2^{10} to 2^{24} , but ns/op rises at least 3 $\times$, in steps near the L2 ($\approx 2^{17}$) and L3 (2^{21} to 2^{23}) limits.
H3b	2026-09-29	maze_scrambled is within 1.3 $\times$ of maze at $n \leq 2^{16}$ and at least 1.5 $\times$ slower at $n \geq 2^{22}$; operation counts are identical.
H4	2026-09-29	In Kruskal, the sort takes $\geq 75\%$ of the time at $n \leq 2^{16}$ and $> 50\%$ up to 2^{21} ; the four $\alpha(n)$ variants' totals are within 10%. Not tested (see 2.6).
H5	2026-09-29	Quick-find and naive_none on chain grow at their textbook rates (log-log slope 0.85 to 1.15); naive_none's path length per find matches $1.5 \cdot n(n-1)/(4n-2)$ within 2%.

Table 3: Hypotheses.

Data seen before the predictions were finalised. While the harness was being built, short smoke tests were run: one repetition of random_mixed and chain at $n = 2^{10}$ to 2^{16} (timings and counts), and five repetitions of maze at $n = 2^{10}$ to 2^{12} . The predictions target cases these did not cover. Results inside those ranges are reported but not counted as confirming a prediction. H2 was revised after the counters showed that its original reasoning ("halving makes fewer writes") was wrong; the original is kept in Appendix B.

H3a, H3b and H4 rest on the difference in cost between a cache hit and a miss to main memory, which is one to two orders of magnitude on current hardware (Drepper 2007).

2.3 Experimental setup

Hardware and software

Item	Value
CPU	Intel Core i9-13900H: 6 performance (P) cores + 8 efficiency (E) cores, 20 threads (Intel Corporation n.d.)
Caches	L2 1.25 MB per P-core and 2 MB per E-core cluster, 11.5 MB in total, matching what Windows reports (PassMark Software n.d.); L3 24 MB shared (Intel Corporation n.d.)
Memory	64 GB
OS	Windows 11 Pro (10.0.26200)
Compiler	MSVC 19.44 (Visual Studio 2022), CMake 3.31, Ninja; Release flags /O2 /DNDEBUG
Plotting	Python 3.12, pandas 3.0, matplotlib 3.11
Benchmark command	build-release\bench.exe --out results\bench.csv --counts-out results\counts.csv --reps 7 --max-log 24 --quadratic-max-log 17 --seed 12345 --min-time-ms 5 (run 30 September 2026, 12:05 to 12:34, 29 minutes)

Table 4: Test machine.

Workloads

n is a power of two. All operation sequences are generated before timing from a fixed seed, so every variant receives the identical sequence.

Workload	Operations	Purpose
random_mixed	n random unions interleaved with n random connected queries	Typical random use
chain	$\text{unite}(i, i+1)$ for all i , then n queries from element 0	Worst case for naive linking (H5)
maze	Randomised Kruskal on a $w \times h$ grid ($w \cdot h = n$): one unite per wall, shuffled	The application
maze_scrambled	The same maze with cells randomly relabelled	Isolates memory locality (H3b)

Table 5: Workloads.

Measurement methodology

- Only the operation loop is timed (`std::chrono::steady_clock`). Workload generation, the construction of each union-find object and all output happen outside the timed region.
- Each variant gets one discarded warm-up run. A sample then repeats the run (a fresh object each time, built untimed) until its timed total is at least 5 ms, and reports ns per operation. Each point is the median of the samples, with the interquartile range (IQR) as error bars.
- The process is pinned to one P-core at raised priority.
- Quick-find and naive_none can be quadratic, so they were run only up to $n = 2^{17}$. All other variants were run from 2^{10} to 2^{24} . Each point has 7 samples.
- Operation counts come from a separate untimed run of the counting build, whose checksum must match.

Measurement noise and how it was reduced

The first multi-repetition runs were too noisy for the planned comparisons. At $n = 2^{10}$ to 2^{12} , the IQR across 5 repetitions was up to 105% of the median, and the median spread was 16%, so a “within 10%” prediction could not be tested. Three likely causes were identified:

- **Very short timed runs.** At $n = 2^{10}$, one run lasts about 10 to 20 μ s, so a single OS interruption or timer tick is a large fraction of it.
- **No warm-up.** The first repetition ran with cold caches and before the CPU clock had ramped up.
- **Hybrid cores.** The OS can move the thread between P-cores and slower E-cores during a run, and each move also loses the warm L1 and L2 caches.

The fixes were: a discarded warm-up run, a minimum timed duration per sample (5 ms, reached by repeating the run), and pinning to a P-core at raised priority. Table 6 shows the effect on the same runs.

IQR / median (5 reps, $n = 2^{10}$ to 2^{12})	Median	90th percentile	Worst	Points < 5%
Before	16.3%	61.8%	105.2%	30%
After	1.0%	4.3%	17.3%	92%

Table 6: Run-to-run spread before and after the noise fixes.

The contribution of each fix was not measured separately; the three were applied together.

In the final run (7 samples per point, 2^{10} to 2^{24} , 544 points), the median spread was 2.0%, the 90th percentile 8.1%, and 93% of points were under 10%. The worst point was 42.4% (chain, size_none, $n = 2^{20}$). Differences smaller than about 10% between two variants are therefore treated as inconclusive.

Threats to validity

- **Warm caches at small n.** Repeating a run to reach the minimum sample time reuses caches that the previous run warmed, so small-n results measure warm-cache cost. This is the same for every variant.
- **One machine and one compiler.** Results are for a single CPU and MSVC. MSVC does not apply `-march=native`-style tuning. `std::shuffle` and the `<random>` distributions are implementation-defined, so the same seed produces different sequences on GCC or Clang.
- **Pinning relies on the OS.** The P-core is chosen from the efficiency class Windows reports. It was not verified independently.
- **Checksum in the loop.** The checksum adds a multiply-add per operation to every variant. It prevents the compiler from removing the loop and is the same for all variants.

2.4 Results

All variants produced identical checksums for every workload and size, and the counting build matched the timed build in all 512 counted cases (quick-find has no counting build), so every variant computed the same answers.

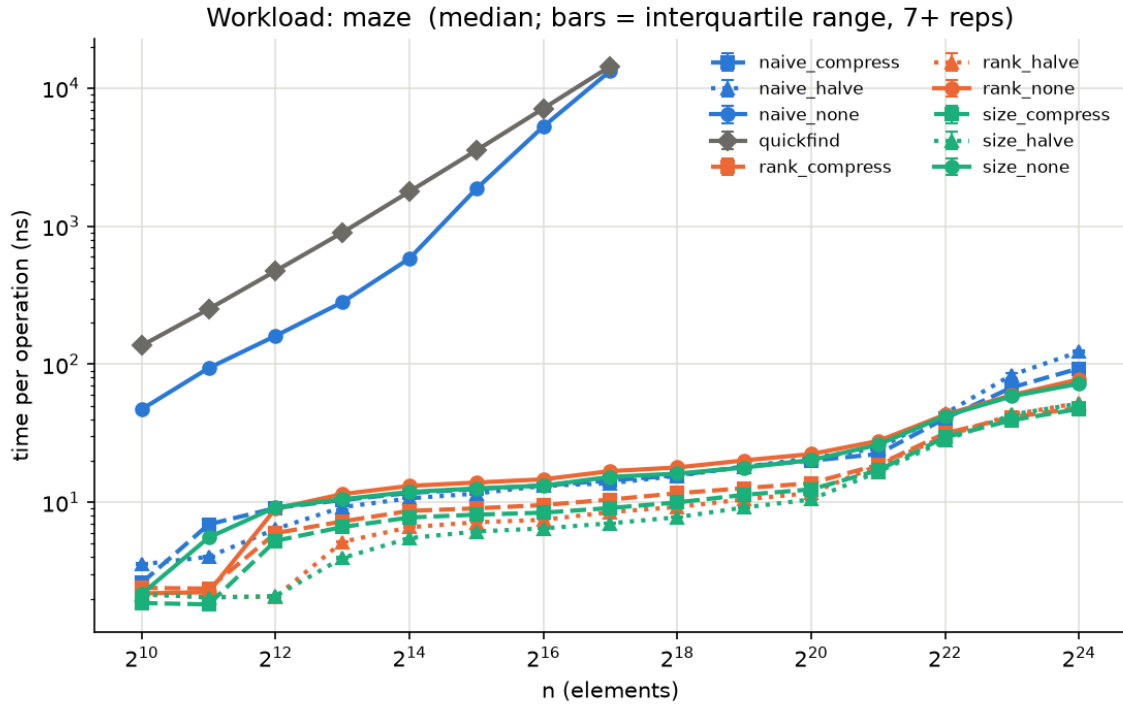


Figure 1: maze: median ns per operation against n for every variant (log-log). Error bars show the IQR over 7 samples. Quick-find and naive_none stop at 2^{17} .

Figure 1 shows two groups. Quick-find and naive_none rise steeply, reaching about 1.4×10^4 ns per operation at 2^{17} . The other seven variants stay between about 2 and 120 ns over the whole range. The same grouping appears on random_mixed (Figure 2).

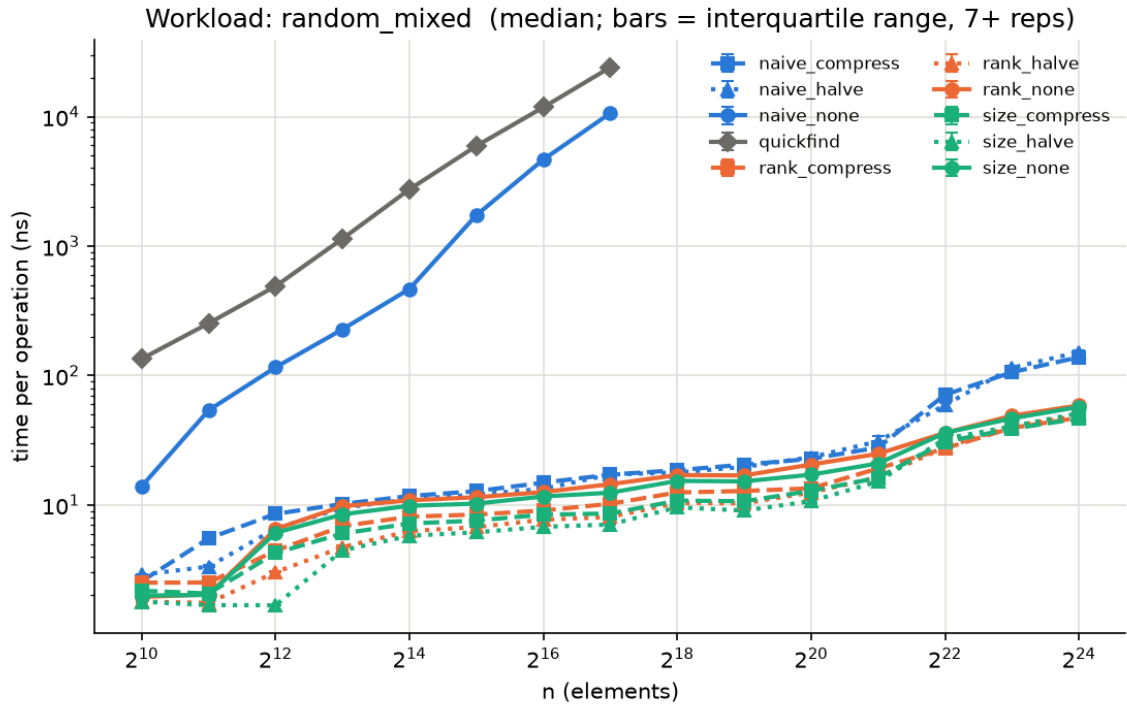


Figure 2: random_mixed: median ns per operation against n ; error bars show the IQR over 7 samples.

Time per operation vs work per operation (H3a)

rank_compress	2 ¹⁰	2 ¹⁴	2 ¹⁷	2 ²⁰	2 ²²	2 ²⁴	Rise 2 ¹⁰ →2 ²⁴
random_mixed ns/op	2.51	8.11	10.21	13.45	27.33	47.16	18.8×
maze ns/op	2.40	8.66	10.46	13.74	31.58	48.22	20.1×

Table 7: Median ns per operation for rank_compress.

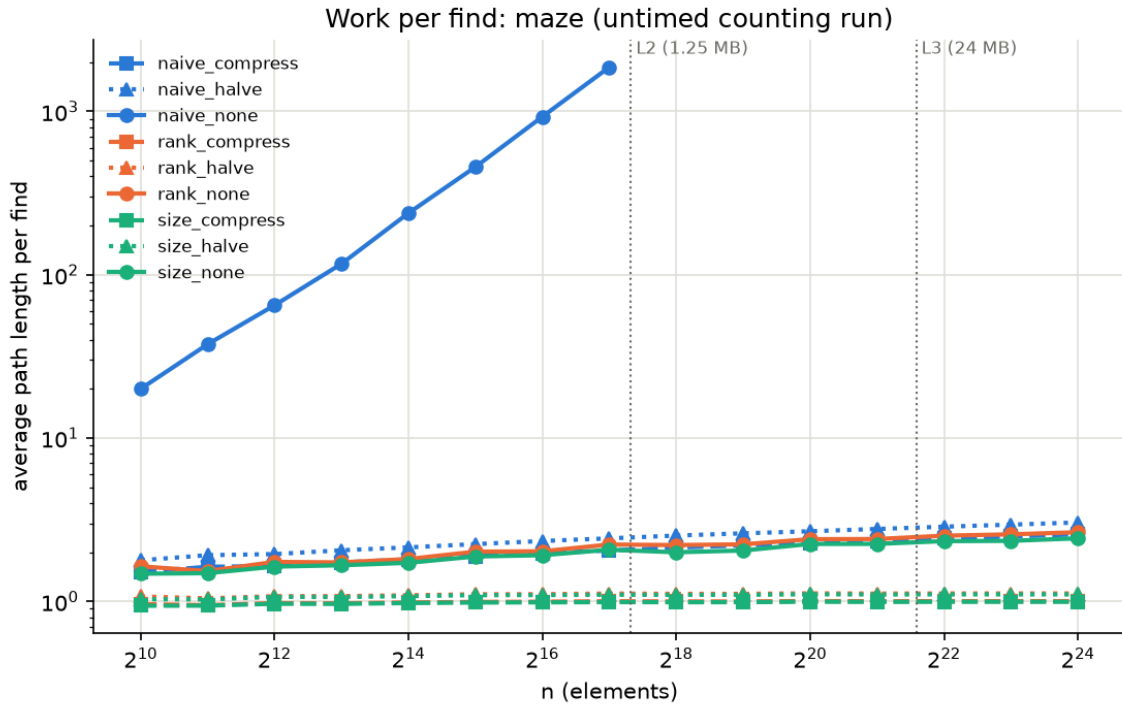


Figure 3: maze: average path length per find against n , from the untimed counting run. The dotted lines mark where the parent and rank arrays (8 bytes per element) exceed the 1.25 MB L2 and the 24 MB L3.

Over the same range, the path length per find for rank_compress stayed between 0.61 and 0.63 on random_mixed and between 0.96 and 1.00 on maze (Figure 3). Time per operation rose in three phases: by 3.2 to 3.7 $\times$ from 2^{11} to 2^{14} , by only 1.03 to 1.22 $\times$ per doubling from 2^{14} to 2^{20} , and by 1.31 to 1.69 $\times$ per doubling from 2^{21} to 2^{23} .

Linking rule and path rule (H1, H2)

Ratio of median ns/op	2^{14}	2^{16}	2^{18}	2^{20}	2^{22}	2^{24}
rank / size, compress, maze	1.12	1.14	1.17	1.11	1.07	1.02
rank / size, compress, random_mixed	1.13	1.08	1.16	1.05	0.89	1.02
half / compress, rank, maze	0.76	0.78	0.80	0.85	0.95	1.09
half / compress, rank, random_mixed	0.77	0.85	0.83	0.93	1.02	1.04

Table 8: Ratios between variants. Values above 1 mean the first variant is slower.

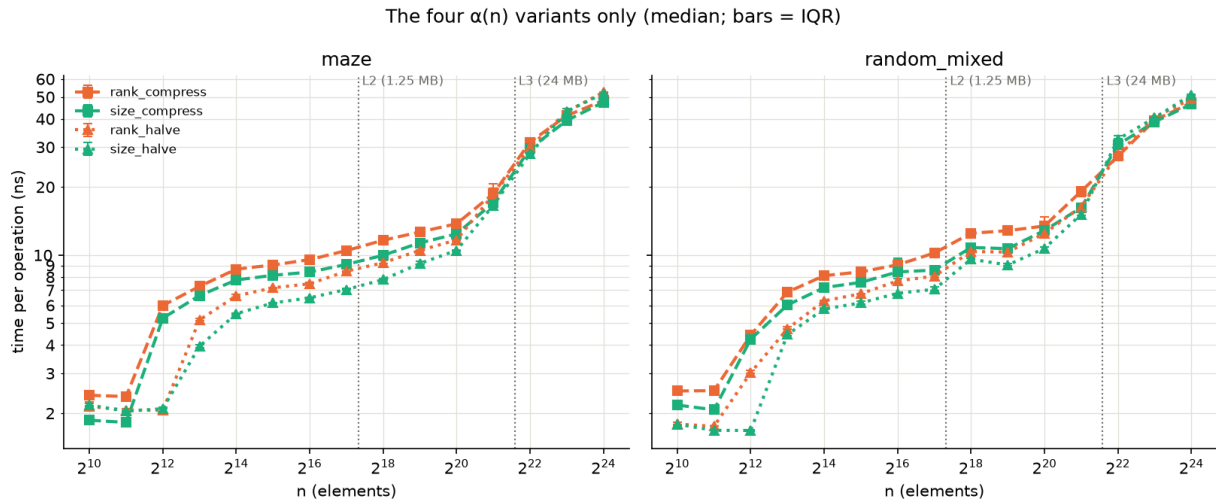


Figure 4: The four $\alpha(n)$ variants only: median ns per operation against n , with IQR error bars. Rank (orange) sits above size (green), and halving (dotted) below compression (dashed), until the lines converge beyond the L3 limit.

Rank linking was slower than size linking at 12 of 12 sizes from 2^{13} on maze and at 7 of 8 sizes from 2^{17} on random_mixed, by up to 16.5% and 20.0% respectively for compression, and by up to 30.8% for halving (maze, 2^{13}). The gap fell below 10% from 2^{22} , except on random_mixed at 2^{22} , where size was slower by 11% (compression) and 14% (halving). The counts differ by about 1%: 0.621 against 0.615 path length per find at 2^{20} on random_mixed.

Halving was 7 to 40% faster than full compression from 2^{13} to 2^{20} (mostly 15 to 25%), although it made about $3.4\times$ as many parent writes (0.50 against 0.15 per find). The ratio moved toward 1 as n grew, and at 2^{23} and 2^{24} full compression was 3 to 10% faster. Halving's paths were about 5% longer (0.655 against 0.621 per find at 2^{20} on random_mixed).

Memory layout (H3b)

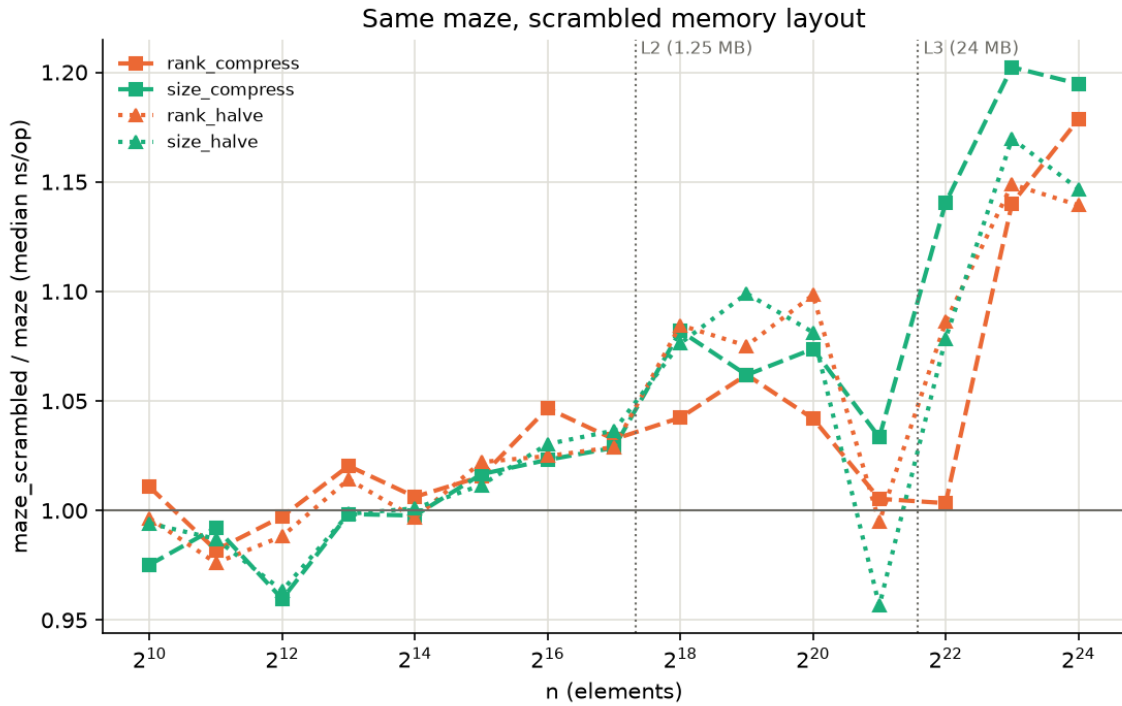


Figure 5: Median ns per operation of `maze_scrambled` divided by `maze`, for the four $\alpha(n)$ variants.

Path length, writes, maximum path and checksum were identical for `maze` and `maze_scrambled` in all 128 variant–size pairs, as expected. The time ratio stayed between 0.96 and 1.10 up to 2^{21} , and between 1.00 and 1.20 from 2^{22} to 2^{24} (Figure 5).

Baselines (H5)

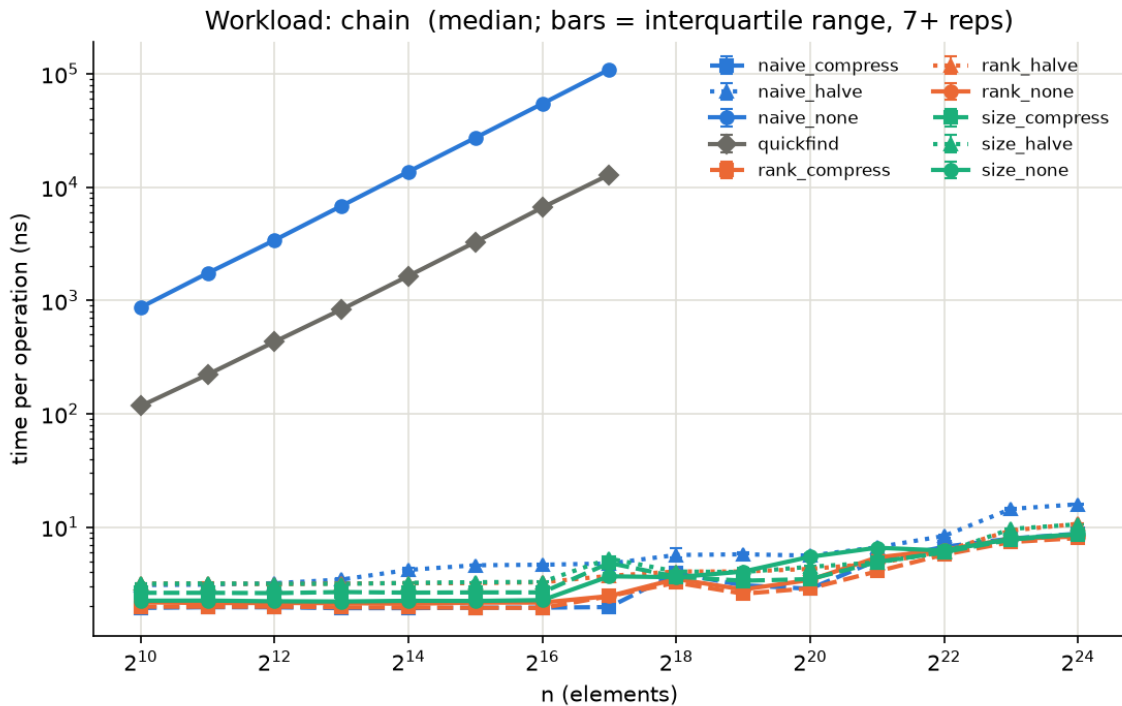


Figure 6: chain: median ns per operation against n .

Measurement	Value
Quick-find, log-log slope of ns/op, 2^{14} to 2^{17}	1.039 (random_mixed), 1.005 (maze)
naive_none on chain, log-log slope of ns/op, 2^{10} to 2^{17}	0.996
naive_none on chain, path length per find vs $1.5 \cdot n(n-1)/(4n-2)$	within 0.63% at every n (49,139 vs 49,152 at 2^{17})
At 2^{17} : naive_none $\div$ slowest other variant	619 $\times$ (random_mixed), 793 $\times$ (maze)
rank_none and size_none: max_path $\leq$ floor(log2 n)	0 violations; largest max_path = $0.6 \cdot \log_2 n$

Table 9: Growth rates of the baselines.

2.5 Analysis

#	Outcome	Evidence
H1	Refuted	Rank slower than size by up to 16.5% (maze) and 20.0% (random_mixed) with compression, and up to 30.8% with halving, between 2^{13} and 2^{20} ; mostly within 10% from 2^{22} (Table 8).
H2	Supported	Halving 7 to 40% faster at small and medium n; ratio 0.95 to 1.10 at $n \geq 2^{22}$, so the gap shrinks (Table 8).
H3a	Mostly supported	Path length flat; time up 18.8 $\times$ and 20.1 $\times$; largest per-doubling rises at 2^{21} to 2^{23} (L3). The predicted L2 step was small, and an unpredicted rise of 3.2 to 3.7 $\times$ occurred from 2^{11} to 2^{14} (Table 7, Figure 3).
H3b	Refuted (control holds)	Counts identical, as required. Scrambling cost at most 1.20 $\times$ beyond L3, below the predicted 1.5 $\times$ (Figure 5).
H4	Not tested	The Kruskal workload was not implemented in the time available.
H5	Supported	Slopes 0.996 to 1.039; path length within 0.63% of the formula; ordering by bound holds by a factor of at least 619 (Table 9).

Table 10: Hypothesis outcomes.

Where theory and practice agree

When the work per operation grows with n , the measured time follows the textbook rate closely (H5). Quick-find relabels $\Theta(n)$ entries per union, and naive linking on the chain walks $\Theta(n)$ parents per query. Both access memory in order, so the hardware charges a roughly constant cost per unit of work, and the log-log slopes are within 4% of 1. The chain's path length matches the exact formula to within 0.63%. The bound also correctly predicts the ordering: every $\alpha(n)$ variant is at least 619 $\times$ faster than naive_none at 2^{17} .

Where they diverge, and why

For the $\alpha(n)$ variants, the work per operation is effectively constant: path length per find stays within 0.61 to 0.63 on random_mixed and 0.96 to 1.00 on maze across a 16,384-fold range of n . Time per operation nevertheless rises about 19 to 20 $\times$. Since the counted work does not change, the rise must come from the cost of each memory access. The largest per-doubling increases occur from 2^{21} to 2^{23} , where the parent and rank arrays (16 to 64 MB) outgrow the 24 MB L3, and misses to main memory cost one to two orders of magnitude more than hits (Drepper 2007). The analyses of Tarjan (1975) and Tarjan and van Leeuwen (1984) count pointer operations at unit cost, so it cannot see this. This is consistent with the cache explanation, but cache misses were not measured directly (no hardware counters were used).

Two parts of the prediction did not hold. The L2 step was weak (1.10 to 1.22 $\times$), and there was an unpredicted rise of 3.2 to 3.7 $\times$ between 2^{11} and 2^{14} . At 2^{12} the parent and rank arrays alone

take 32 KB of the 48 KB L1 data cache of a P-core (PassMark Software n.d.), and the operation list is streamed alongside them. L1 pressure may contribute, but this estimate does not explain why the jump starts at 2^{12} rather than 2^{11} , and it was not tested.

H3b tested the locality part of this explanation directly and refuted the predicted size of the effect. Relabelling the cells changed no count, yet cost at most 20% even beyond L3. In maze the two cells of a wall are adjacent in memory, but the walls are processed in random order, so consecutive operations still touch unrelated parts of the grid, and a find ends at a root that can be anywhere in its component. The ordered layout therefore gives little locality to lose. The rise in H3a is better explained by the size of the working set than by the layout of this workload.

Constant factors between the $\alpha(n)$ variants

The bounds do not distinguish rank from size or halving from compression, and at large n the measurements agree: differences shrink to within about 10% from 2^{22} (Table 8). Below that, constant factors dominate. Halving is up to 40% faster despite 3.4× as many writes, which is consistent with its single pass mattering more than its stores while the data is in cache. Once most accesses miss, time tracks path length more closely: halving's paths are about 5% longer, and at 2^{23} and 2^{24} it is 3 to 10% slower than full compression.

Rank linking's deficit of up to 20% (compression) and 31% (halving) against size linking (H1) is not explained by the counts, which differ by about 1%. One candidate is the extra data-dependent branch in rank linking (incrementing the rank only on a tie), which size linking does not have. This was not tested.

2.6 Limitations and further work

- H4 (Kruskal) was not tested: the workload could not be implemented before the deadline (see 3.2).
- No hardware performance counters were used, so the cache explanations in 2.5 are inferred from timing and counts, not measured. Counting cache misses per operation (for example with Intel VTune) would test them directly.
- The cause of the rank-vs-size gap is untested. A branchless rank update would test the branch explanation.
- The rise from 2^{11} to 2^{14} is unexplained; testing it would need finer n steps around the L1 size.
- The worst case for rank linking without compression (binomial-tree unions, depth $\log_2 n$) is not a workload; random inputs reached at most $0.6 \cdot \log_2 n$.
- Only one union-to-query ratio is tested (1:1 in `random_mixed`), and path splitting (Tarjan & van Leeuwen 1984) is not implemented.
- Results come from one machine and one compiler, with warm caches at small n (see Threats to validity).

3. What I learned

3.1 What surprised me

The result that surprised me most was H3b. I expected the scrambled maze to be at least $1.5\times$ slower than the ordered one once the arrays no longer fit in L3, because the ordered layout puts the two cells of a wall next to each other in memory. It was at most $1.20\times$ slower. The reason is that randomised Kruskal visits the walls in random order: consecutive operations touch unrelated parts of the grid, and each find ends at a root that can be anywhere in its component. The ordered layout had very little locality to lose. I had assumed that neighbouring data means cache-friendly access without checking the order in which the data is actually accessed.

The second surprise was how exactly theory and practice agreed in some places and how far apart they were in others. Quick-find's time per operation doubled when n doubled (log-log slopes of 1.04 and 1.01), and naive linking on the chain matched the exact path-length formula to within 0.63%. Yet rank_compress became about $20\times$ slower between 2^{10} and 2^{24} , while its path length per find stayed between 0.61 and 0.63. The $\alpha(n)$ bound was right about the work and said nothing about the time, because it counts pointer operations at unit cost, and the hardware does not.

3.2 What I initially got wrong

Three predictions were wrong. First, I expected rank and size linking to be within 10% of each other once compression was used, since they have the same bound and almost the same path length. Rank was consistently slower, by up to 20% with compression and 31% with halving, and the counts (0.621 against 0.615 path length per find) do not explain the gap. I still do not know the cause.

Second, the first version of H2 reasoned that halving would make fewer parent writes than full compression. The counters showed the opposite, about 0.50 writes per find against 0.15, because halving stores to a node even when it already points at the root, and with rank or size linking almost every path is that short. Halving was still faster at small and medium n , so making fewer writes was not the reason it wins.

Third, and most important for the study, I initially trusted the benchmark. The first multi-repetition results looked reasonable as a plot of medians, but when I added error bars the spread between repetitions was up to 105% of the median. Without the warm-up run, the minimum sample time and pinning to a P-core, none of the 10% comparisons in H1 and H2 would have meant anything. After the fix the median spread was 2.0%.

I also got the planning wrong. I wrote hypotheses for five experiments, including Kruskal's MST (H4), before checking how much each would cost to build and run. The noise problem and the maze workloads took longer than expected, and the final run alone took 29 minutes (12:05 to 12:34 on 30 September 2026). With the deadline close, I had to choose between implementing the Kruskal workload and finishing the analysis of the experiments I had already run. I chose the analysis, so H4 is reported as not tested rather than tested badly. If I did this again, I would build and smoke-test every workload first, and only then write the full set of hypotheses, dropping any I could not run in time.

3.3 What I understand now

At the start I could not have explained why the inverse-Ackermann bound applies to some variants and not others. I now understand that it needs both a linking rule and a path rule: path compression alone gives $O(\log_{1+m/n} n)$ and union by rank alone gives $O(\log n)$. I also understand why $\alpha(n)$ cannot be observed directly. It is at most 4 for any practical n , so on a plot it is indistinguishable from a constant. What can be measured is its prediction that the work per operation stays flat, and the counters confirmed that.

Maze generation made Kruskal's algorithm concrete for me. Kruskal's cycle test is exactly the return value of `unite`: an edge is added only if its endpoints are in different sets. Randomised Kruskal on a grid is Kruskal with random edge weights, so shuffling the walls takes the place of sorting the edges. That is why the result is always a spanning tree with $w \cdot h - 1$ passages: a removal can never close a loop, and the grid is connected. A mutation test showed which check does the work. Removing the `if` around `unite` still produced exactly $w \cdot h - 1$ removals because of the early stop, so the connectivity check caught the bug and the count check did not. It also clarified what H4 was about: in full Kruskal the sort is the same for every variant, so the choice of union-find should matter much less than it does in the maze. I did not have time to test this.

4. AI use

I used Claude Code (Claude Opus 5.5) during the development of this project between 25 and 30 September 2026.

4.1 How AI was used

AI was mainly used as a development assistant throughout the project. I used it to help organise the overall structure of the repository, suggest approaches for parts of the algorithms, generate or refine some implementation code, and assist with writing tests. It was also useful for debugging build errors and explaining unfamiliar tools or concepts when I encountered them.

For example, I used Claude to help with the union-find implementations, the quick-find baseline, test cases, the benchmark harness, the maze generator and parts of the CMake configuration. I reviewed the generated code, ran the builds and tests myself, and made changes where the output did not behave as expected.

I also used AI when developing the benchmark methodology. This included suggestions for reducing measurement noise, such as adding warm-up runs, increasing the minimum sample time and pinning the benchmark to a performance core. I then ran the final benchmarks myself and checked that the results were consistent.

4.2 Analysis, writing and debugging

Claude was also used to help analyse the benchmark results and produce the graphs used in the report. It assisted with checking the output CSV files, comparing results against the hypotheses, and generating the plotting script.

For the written report, I mainly used AI to help with structure, wording and organising my notes into clearer sections. With my tutor's permission, some sections were initially drafted from my session notes and AI log, after which I reviewed and edited them myself. I also used it to help check references and explain concepts such as CMake, library testing and the complexity of the union-find algorithms.

Overall, AI was used as a supporting tool for implementation, debugging, testing, analysis and report structure. I remained responsible for deciding the direction of the project, choosing what approaches to keep or discard, running and verifying the code and benchmarks, reviewing the results, and making the final commits.

4.3 Where the AI was wrong, unhelpful, or misleading

1. It said no compiler was installed. At the start it reported that the machine had no CMake or C++ compiler, and wrote the first version of the code without being able to compile it. In fact Visual Studio 2022, with the C++ tools, CMake and AddressSanitizer, was installed but not on the default PATH. Checking with `vswhere` found it, and the code was then built and tested.
2. Its build instructions broke my build. It told me to use the Start-menu Developer PowerShell, which defaults to 32-bit. My first configure failed with LNK4272 (x86 libraries against an x64 target), made worse because a build folder the AI had configured earlier had cached a different compiler. It was fixed by launching the 64-bit shell explicitly and deleting the build folder, and the README was corrected.
3. It looked at data before the predictions were final. To check a chart layout it opened a real maze plot ($n = 2^{10}$ to 2^{12}) instead of using made-up data, so those results were no longer blind. It

disclosed this, and the method note in section 2.2 excludes that range from confirming any prediction.

4.4 What I understood versus what I took on trust

Understood: How to build and run the project, and what the tests, the demo and the benchmark each show. The core of the implementation in `union_find.hpp`: how `find` and `unite` work for each linking and path rule, why compression uses two iterative passes instead of recursion, and how the rank invariant is established in `unite` and relied on for the $\log n$ height bound. Why randomised Kruskal always produces a spanning tree, and why removing the `if` around `unite` is caught by the connectivity check and not the wall count. And the main result: the bound correctly predicts the work per operation but not the time, because of the memory hierarchy.

Taken on trust: I relied on the tests, the demo and the benchmark cross-checks, rather than a line-by-line review, for the supporting code: the benchmark harness details, the CPU-pinning code in `platform.hpp`, the counting instrumentation and the plotting script. I have not read Tarjan's or Tarjan and van Leeuwen's proofs; I rely on the stated bounds and on checking the references, not on working through the analysis. The cache explanations in section 2.5 are inferred from timing and operation counts, not measured with hardware counters. I also take on trust that the pinning code really selects a P-core, since it relies on what Windows reports, and that 7 samples per point are enough, beyond what the spread figures in section 2.3 show.

References

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. & Stein, C. 2022, *Introduction to algorithms*, 4th edn, MIT Press, Cambridge, MA.
- Drepper, U. 2007, *What every programmer should know about memory*, Red Hat, Inc., <<https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>>.
- Fredman, M.L. & Saks, M.E. 1989, 'The cell probe complexity of dynamic data structures', *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, ACM, New York, pp. 345–354, doi:10.1145/73007.73040.
- Galler, B.A. & Fischer, M.J. 1964, 'An improved equivalence algorithm', *Communications of the ACM*, vol. 7, no. 5, pp. 301–303, doi:10.1145/364099.364331.
- Hopcroft, J.E. & Ullman, J.D. 1973, 'Set merging algorithms', *SIAM Journal on Computing*, vol. 2, no. 4, pp. 294–303, doi:10.1137/0202024.
- Intel Corporation n.d., *Intel Core i9-13900H processor (24M cache, up to 5.40 GHz): product specifications*, Intel, <<https://www.intel.com/content/www/us/en/products/sku/232135/intel-core-i913900h-processor-24m-cache-up-to-5-40-ghz/specifications.html>>.
- Kruskal, J.B. 1956, 'On the shortest spanning subtree of a graph and the traveling salesman problem', *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, doi:10.1090/S0002-9939-1956-0078686-7.
- PassMark Software n.d., *Intel Core i9-13900H benchmark*, PassMark CPU Benchmarks, <<https://www.cpubenchmark.net/cpu.php?cpu=Intel+Core+i9-13900H&id=5210>>.
- Patwary, M.M.A., Blair, J. & Manne, F. 2010, 'Experiments on union-find algorithms for the disjoint-set data structure', in P. Festa (ed.), *Experimental algorithms: 9th International Symposium, SEA 2010*, Lecture Notes in Computer Science, vol. 6049, Springer, Berlin, pp. 411–423, doi:10.1007/978-3-642-13193-6_35.
- Tarjan, R.E. 1975, 'Efficiency of a good but not linear set union algorithm', *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, doi:10.1145/321879.321884.
- Tarjan, R.E. & van Leeuwen, J. 1984, 'Worst-case analysis of set union algorithms', *Journal of the ACM*, vol. 31, no. 2, pp. 245–281, doi:10.1145/62.2160.

Appendix A: Reproducing the results

On Windows, first open a 64-bit Visual Studio 2022 developer shell:

```
& "C:\Program Files\Microsoft Visual Studio\2022\Professional\Common7\Tools\Launch-VsDevShell.ps1" -Arch amd64 -HostArch amd64
```

Build and test (Debug, with sanitizers):

```
cmake -S . -B build-debug -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build-debug
ctest --test-dir build-debug --output-on-failure
```

Benchmark and plots (Release):

```
cmake -S . -B build-release -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build-release
```

```
build-release\bench.exe --out results\bench.csv --counts-out results\counts.csv --reps 7 --  
max-log 24 --quadratic-max-log 17 --seed 12345 --min-time-ms 5  
python scripts\plot.py
```

Appendix B: Hypotheses

The full statement of each hypothesis (prediction, reasoning, refutation criteria and experiment), the note on data seen in advance, and the revision history of H2 are in the accompanying document `hypotheses.docx`.